Министерство образования Республики Беларусь

Учреждение образования «Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Модели решения задач в ИС»

Тема: «Адаптивный шаг обучения в однослойном линейном персептроне (метод наискорейшего спуска)»

Выполнил:

Студент 3 курса

Группы ИИ-26 (2)

Суховило В.С.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: изучить алгоритм оптимизации градиентного спуска с использованием адаптивного шага обучения. Реализовать модифицированный персептрон, в котором параметр скорости обучения t вычисляется на основе минимизации квадратичной формы ошибки для каждой итерации. Сравнить скорость сходимости с классическим алгоритмом из Лабораторной работы №1. Вариант сохраняется.

Задачи лабораторной работы:

1. Модифицировать алгоритм последовательного обучения (из Лаб №1) таким образом, чтобы на каждой итерации t значение α вычислялось автоматически на основе текущего входного вектора x по формул (см раздел 2.9).
2. Применить вычисленный α(t) для обновления весов ij и порогов Tj согласно дельта-правилу.
3. Используя данные своего варианта, провести два эксперимента:
   - Обучение с фиксированным шагом (например, α=0.1 или α=1p).
   - Обучение с адаптивным шагом по Теореме 2.1 (формула 2.36).

Критерий остановки в обоих случаях – достижение заданной суммарной ошибки Es≤Ee.

4. Построить графики обучения Es(p), где p – номер эпохи, для обоих экспериментов на одних осях координат.
5. Выполнить графическую визуализацию разделяющей линии для адаптивного метода.
6. Реализовать режим функционирования сети:
   - пользователь задаёт произвольный входной вектор,
   - сеть вычисляет выходной класс,
   - соответствующая точка отображается на графике,
7. Написать вывод по выполненной работе. Оценить, насколько адаптивный шаг сокращает количество эпох обучения по сравнению с фиксированным. Обязательно сравнение результатов 1 и 2 лабораторных работ.

Допускается применение **математических** и **графических** библиотек
ML-библиотеки и **ML-фреймворки** использовать **нельзя** (например: scikit-learn, TensorFlow, PyTorch - запрещены)

Вариант 5:

5.

x_1	x_2	e
2	6	0
-2	6	1
2	-6	0
-2	-6	0

Код программы:

```
import numpy as np
import matplotlib
matplotlib.rcParams['toolbar'] = 'None'
import matplotlib.pyplot as plt

X = np.array([
    [2, 6],
    [-2, 6],
    [2, -6],
    [-2, -6]
], dtype=float)

E = np.array([0, 1, 0, 0], dtype=int)
T = np.where(E == 0, -1, 1).astype(float)

X1_MIN, X1_MAX = -3.0, 3.0
X2_MIN, X2_MAX = -7.0, 7.0

def step(u):
    return 1 if u >= 0 else 0

def net(x, w, b):
    return float(np.dot(w, x) + b)

def train_fixed_alpha(X, T, alpha=0.1, Ee=1e-4, max_epochs=5000):
    w = np.zeros(X.shape[1], dtype=float)
    b = 0.0
    Es_hist = []
    epochs = 0

    while epochs < max_epochs:
        Es = 0.0
        for x, t in zip(X, T):
            y = net(x, w, b)
            err = t - y
            Es += err ** 2
            w += alpha * err * x
            b += alpha * err
        Es_hist.append(Es)
        epochs += 1
        if Es <= Ee:
            break

    return w, b, np.array(Es_hist), epochs

def train_adaptive_alpha(X, T, Ee=1e-4, max_epochs=5000):
    w = np.zeros(X.shape[1], dtype=float)
    b = 0.0
```

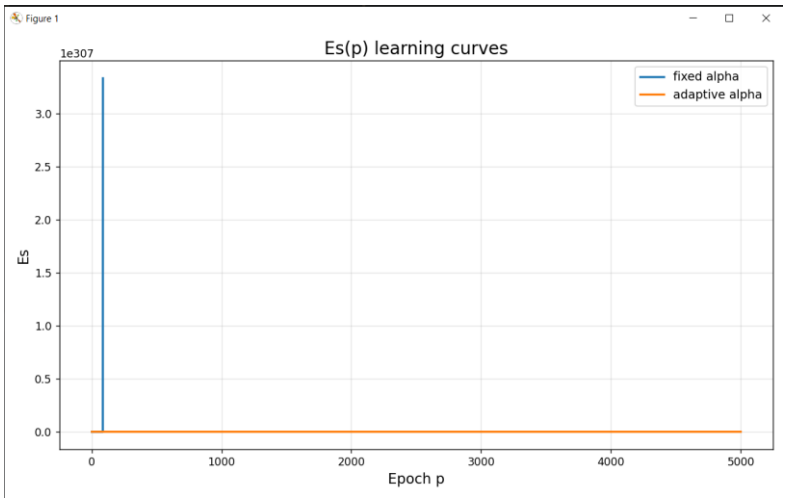
```
    Es_hist = []
    epochs = 0

    while epochs < max_epochs:
        Es = 0.0
        for x, t in zip(X, T):
            y = net(x, w, b)
            err = t - y
            Es += err ** 2

            denom = np.dot(x, x) + 1.0
            alpha_t = 1.0 / denom if denom != 0 else 0.0

            w += alpha_t * err * x
            b += alpha_t * err

        Es_hist.append(Es)
        epochs += 1
        if Es <= Ee:
            break

    return w, b, np.array(Es_hist), epochs

def forward_class(X, w, b):
    return np.array([step(net(x, w, b)) for x in X], dtype=int)

def plot_es(hist_fixed, hist_adapt):
    plt.figure(figsize=(10, 6))
    plt.plot(np.arange(1, len(hist_fixed) + 1), hist_fixed, linewidth=2, label="fixed alpha")
    plt.plot(np.arange(1, len(hist_adapt) + 1), hist_adapt, linewidth=2, label="adaptive alpha")
    plt.title("Es(p) learning curves", fontsize=16)
    plt.xlabel("Epoch p", fontsize=13)
    plt.ylabel("Es", fontsize=13)
    plt.grid(True, alpha=0.35)
    plt.legend(fontsize=11)
    plt.tight_layout()

def plot_boundary_and_points(X, E, w, b, user_point=None):
    plt.figure(figsize=(10, 8))
    plt.xlabel("x1", fontsize=13)
    plt.ylabel("x2", fontsize=13)
    plt.title("Adaptive method: decision boundary", fontsize=16)
    plt.grid(True, alpha=0.35)

    xx, yy = np.meshgrid(
        np.linspace(X1_MIN, X1_MAX, 300),
        np.linspace(X2_MIN, X2_MAX, 300)
    )
    zz = w[0] * xx + w[1] * yy + b
```

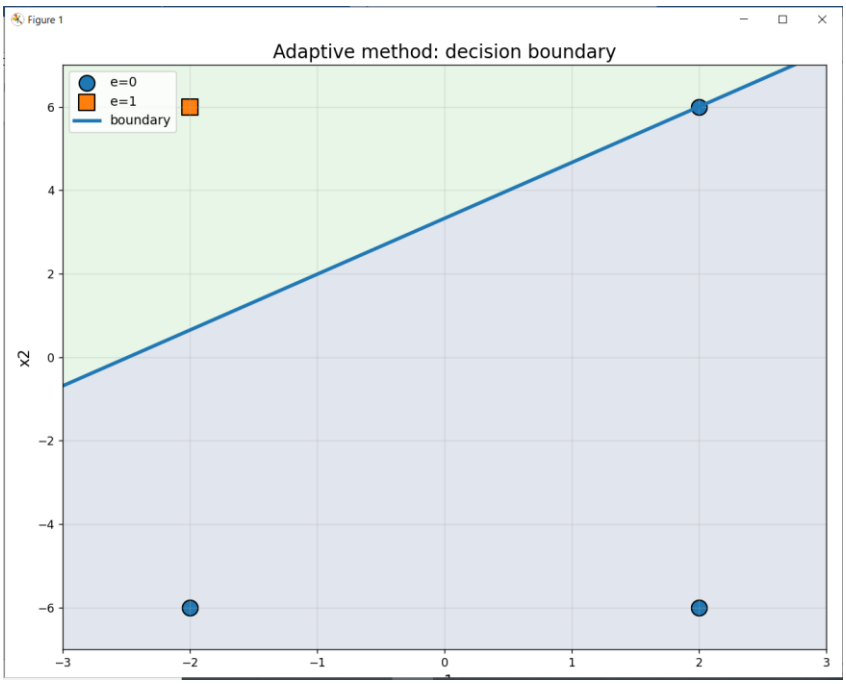
```
    plt.contourf(xx, yy, zz, levels=[-1000, 0, 1000], alpha=0.15)

    X0 = X[E == 0]
    X1c = X[E == 1]

    if len(X0) > 0:
        plt.scatter(X0[:, 0], X0[:, 1], marker="o", s=180, edgecolors="black", linewidths=1.2, label="e=0")
    if len(X1c) > 0:
        plt.scatter(X1c[:, 0], X1c[:, 1], marker="s", s=180, edgecolors="black", linewidths=1.2, label="e=1")

    w1, w2 = w[0], w[1]
    if abs(w2) >= 1e-12:
        x1_vals = np.linspace(X1_MIN, X1_MAX, 300)
        x2_vals = -(w1 * x1_vals + b) / w2
        plt.plot(x1_vals, x2_vals, linewidth=3, label="boundary")
    elif abs(w1) >= 1e-12:
        x1_const = -b / w1
        plt.axvline(x=x1_const, linewidth=3, label="boundary")

    if user_point is not None:
        u = net(np.array(user_point, dtype=float), w, b)
        cls = step(u)
        marker = "x" if cls == 0 else "*"
        plt.scatter([user_point[0]], [user_point[1]], marker=marker, s=220, linewidths=2.2, label=f"user
class={cls}")

    plt.xlim(X1_MIN, X1_MAX)
    plt.ylim(X2_MIN, X2_MAX)
    plt.legend(fontsize=11)
    plt.tight_layout()

def interactive_mode(w, b):
    plt.ion()
    fig, ax = plt.subplots(figsize=(10, 8))
    ax.set_xlabel("x1", fontsize=13)
    ax.set_ylabel("x2", fontsize=13)
    ax.set_title("Interactive mode", fontsize=16)
    ax.grid(True, alpha=0.35)
    ax.set_xlim(X1_MIN, X1_MAX)
    ax.set_ylim(X2_MIN, X2_MAX)

    xx, yy = np.meshgrid(
        np.linspace(X1_MIN, X1_MAX, 300),
        np.linspace(X2_MIN, X2_MAX, 300)
    )
    zz = w[0] * xx + w[1] * yy + b
    ax.contourf(xx, yy, zz, levels=[-1000, 0, 1000], alpha=0.15)

    X0 = X[E == 0]
```

```
    X1c = X[E == 1]

    if len(X0) > 0:
        ax.scatter(X0[:, 0], X0[:, 1], marker="o", s=180, edgecolors="black", linewidths=1.2, label="train e=0")
    if len(X1c) > 0:
        ax.scatter(X1c[:, 0], X1c[:, 1], marker="s", s=180, edgecolors="black", linewidths=1.2, label="train e=1")

    w1, w2 = w[0], w[1]
    if abs(w2) >= 1e-12:
        x1_vals = np.linspace(X1_MIN, X1_MAX, 300)
        x2_vals = -(w1 * x1_vals + b) / w2
        ax.plot(x1_vals, x2_vals, linewidth=3, label="boundary")
    elif abs(w1) >= 1e-12:
        x1_const = -b / w1
        ax.axvline(x=x1_const, linewidth=3, label="boundary")

    ax.legend(fontsize=11)
    fig.tight_layout()
    fig.canvas.draw()
    fig.canvas.flush_events()

    while True:
        s = input("x1 x2 (or q): ").strip()
        if s.lower() in ("q", "quit", "exit"):
            break
        try:
            parts = s.replace(",", " ").split()
            if len(parts) != 2:
                print("Need 2 numbers: x1 x2")
                continue
            x1v = float(parts[0])
            x2v = float(parts[1])
        except:
            print("Bad input. Example: 2 -6")
            continue

        u = net(np.array([x1v, x2v], dtype=float), w, b)
        cls = step(u)
        print(f"class = {cls}")

        marker = "x" if cls == 0 else "*"
        ax.scatter([x1v], [x2v], marker=marker, s=220, linewidths=2.2)
        fig.canvas.draw()
        fig.canvas.flush_events()

    plt.ioff()
    plt.show()

def main():
```

```
    Ee = 1e-4
    alpha_fixed = 0.1

    w_f, b_f, Es_f, ep_f = train_fixed_alpha(X, T, alpha=alpha_fixed, Ee=Ee)
    w_a, b_a, Es_a, ep_a = train_adaptive_alpha(X, T, Ee=Ee)

    pred_f = forward_class(X, w_f, b_f)
    pred_a = forward_class(X, w_a, b_a)

    acc_f = np.mean(pred_f == E)
    acc_a = np.mean(pred_a == E)

    print(f"Fixed alpha={alpha_fixed}: epochs={ep_f}, Es={Es_f[-1]:.6e}, w={w_f}, b={b_f:.6f}, acc={acc_f:.2f}")
    print(f"Adaptive alpha: epochs={ep_a}, Es={Es_a[-1]:.6e}, w={w_a}, b={b_a:.6f}, acc={acc_a:.2f}")

    if ep_a > 0:
        print(f"Speedup (fixed/adaptive) = {ep_f / ep_a:.2f}x")
    else:
        print("Adaptive finished immediately (unexpected).")

    plot_es(Es_f, Es_a)
    plt.show()

    plot_boundary_and_points(X, E, w_a, b_a)
    plt.show()

    interactive_mode(w_a, b_a)

if __name__ == "__main__":
    main()
```

Результат :





В Лабораторной работе №1 использовался классический алгоритм с фиксированным шагом обучения α. Скорость сходимости зависела от выбранного значения α: при малом шаге обучение происходило медленно, при большом — возможны колебания ошибки. Количество эпох до достижения заданного критерия Es ≤ Ee оказалось существенно выше.

В Лабораторной работе №2 применён адаптивный шаг α(t), вычисляемый автоматически на каждой итерации. Это позволило корректировать величину изменения весов в зависимости от текущего входного вектора. В результате суммарная ошибка уменьшалась быстрее, и требовалось меньшее количество эпох для достижения того же критерия остановки.

Вывод:

изучил алгоритм оптимизации градиентного спуска с использованием адаптивного шага обучения. Реализовал модифицированный персептрон, в котором параметр скорости обучения t вычисляется на основе минимизации квадратичной формы ошибки для каждой итерации. Сравнил скорость сходимости с классическим алгоритмом из Лабораторной работы №1.